

Week 3 - Day 11b: Linear and Binary Search

Searching is a fundamental operation in computer science. There are two fundamental algorithms for searching elements in a list: **Linear Search** and **Binary Search**. Both algorithms have their uses depending on the context and data structure.

Before diving into it, we will also review some concepts from the past days:

- Review: **Lists, Loops, Functions, Recursion**
- **Linear Search**
- **Binary Search**

Exercise 1: Reviewing Lists `[]`

1.1 Create a list called `student_grades` that contains 5 integers from 0 to 100.

```
# TODO: Your code here
```

1.2 Print every element in that list `student_grades` using a For-loop

```
# TODO: Your code here
```

1.3 The function `max` finds the maximum element in a list `lst` and is defined as follows

```
def max(lst):  
    max_so_far = lst[0]  
    for element in lst:  
        if element > max_so_far :  
            max_so_far = element  
    return max_so_far
```

Write a non-recursive function `min` that takes a list of integers `lst` and returns the smallest integer in that list, analogous to the `max` function.

```
# TODO: Your code here  
def min(lst):  
    pass  
  
# Use this cell to test your code  
assert min([]) == None  
assert min([1]) == 1  
assert min([5,4,3,2,1]) == 1  
  
print("All tests passed. []")
```

1.4 Now, let's write this recursively. Turn the `max` function into a recursive function and call it `recursive_max`.

```
# TODO: Your code here
def recursive_max(lst):
    pass

# Use this cell to test your code
assert recursive_max([]) == None
assert recursive_max([1]) == 1
assert recursive_max([1, 2, 3, 4, 5]) == 5

print("All tests passed. ☐")
```

1.5 Write a function called `average` that returns the average of numbers in a given list `lst`.

```
# TODO: Your code here
def average(L):
    pass

assert average([]) == None
assert average([1]) == 1
assert average([1, 2, 3, 4, 5]) == 3
assert average([66,67,71,94,95,70,55]) == 74

print("All tests passed. ☐")
```

Exercise 2: Guessing Game [☐☐☐]

Binary search is the optimal strategy for playing the game *Guess The Number* in which the goal is to guess the number x that is known to be in the range a to b . With every guess made, your friend tells you whether your number is less, greater or equal to their guess.

Below is an implementation of the game *Guess The Number* using recursion and binary search.

```
def guess(x, left, right):
    if x > right or x < left:
        print("Invalid number!")
        return -1

    mid = (left+right)//2
    print("My guess is:", mid)

    if mid == x:
        print("Exactly!\n")
        return 1

    if mid > x:
        print("But that is too big!\n")
        return 1 + guess(x, left, mid-1)
```

```

    if mid < x:
        print("But that is too small!\n")
        return 1 + guess(x, mid+1, right)

# Try calling the function for different inputs
guess(x=40, left=1, right=100)

```

2.1 What are the **left** and **right** values for each of the four guesses made in `guess(56, 1, 100)`? Try answering this question without running the code first, and then check your answer after.

2.2 What are the **left** and **right** values for each of the guesses made in `guess(1, 1, 100)`? Try answering this question without running the code first, and then check your answer after.

2.3 How many guesses does it take to find the number `guess(40, 1, 100)`? Try answering this question without running the code first, and then check your answer after.

Exercise 3: Searching in Lists []

We can also use binary search to search for an element in a sorted list. Here is an iterative implementation of binary search using a `while` loop.

```

def iterative_binary_search(x, lst):
    cnt = 0
    left, right = 0, len(lst) - 1

    while left <= right:
        mid = (left + right) // 2
        guess = lst[mid]
        cnt += 1

        if guess == x:
            break
        if guess > x:
            right = mid - 1
        else:
            left = mid + 1
    return cnt

# Try calling the function with the following inputs
iterative_binary_search(x=21, lst=[-14, 0, 12, 21, 44])

```

3.1 Given the array `[-14, 0, 12, 21, 44]` if we are searching for 21 what values of the array are compared to 21 from the array while searching?

3.2 What are the values of **left** and **right** for `iterative_binary_search(35, [-15, -3, 0, 35, 40, 55])`?

3.3 What will happen if we run `iterative_binary_search(4, [1, 9, 8, -1, 4, 11])`? Why does this not work? How would you fix it?

3.4 If you have a sorted array with 20 numbers, what is the maximum (worst case) number of comparisons you have to do to find a given number in a binary search?

3.5 The above function only works if the number you are searching for is in the array. How would you fix it to work when you are searching for a number that is not in the array?

3.6 Implement the fixed version of the function.

```
# TODO: Your code here
```

Exercise 4: Linear and Binary Search [] []

4.1 Write a function `find` that takes a list `lst` and value `x` and returns the index of `x` in `lst`. Return `None` if it doesn't exist.

NB: Do not assume that the list is sorted.

```
# TODO: Your code here
def linear_search(x, lst):
    pass

# Use this cell to test your code
assert linear_search(0, []) == None
assert linear_search(0, [3, 2, 1]) == None
assert linear_search(0, [0, 1, 2, 3]) == 0

print("All tests passed. [ ]")
```

4.2 Now, assume that the list is sorted. Write a function called `binary_search(x, lst)` that determines the index of `x` in `lst`. If the element is not present, return `None`. You can choose to implement it iteratively or recursively.

```
# TODO: Your code here
def binary_search(x, lst):
    pass

# Use this cell to test your code
assert binary_search(0, []) == None
assert binary_search(0, [1, 2, 3]) == None
assert binary_search(0, [0, 1, 2, 3]) == 0

print("All tests passed. [ ]")
```

Exercise 5: String Search [] []

We have seen string comparisons before. String comparisons follow certain rules. For example, `'a' < 'b'`, `'aa' < 'ab'` and `'a' < 'ab'`. We will use these rules to find the index of strings that fulfil certain conditions in a list of strings.

5.1 Write a function `find_smaller` that on input a list `lst` and an element `x`, outputs position `i` so that `L[i] <= x` if there is such an `i`, and otherwise outputs `None`. If there is more than one such `i`, the function may return any one of them. For example,

```
find_smaller("Liverpool", ["Manchester United", "Arsenal", "Chelsea",
                             "Everton"]) == 1 # 2, 3 also possible
find_smaller("Chelsea", ["Manchester United", "Arsenal", "Chelsea",
                           "Everton"]) == 1 # 2 also possible
find_smaller("Arsenal", ["Manchester United", "Arsenal", "Chelsea",
                           "Everton"]) == 1

# TODO: Your code here
def find_smaller(x, lst):
    pass

# Use this cell to test your code
assert find_smaller("Liverpool", ["Manchester United", "Arsenal",
                                    "Chelsea", "Everton"]) in [1, 2, 3]
assert find_smaller("Chelsea", ["Manchester United", "Arsenal",
                                 "Chelsea", "Everton"]) in [1, 2]
assert find_smaller("Arsenal", ["Manchester United", "Arsenal",
                                 "Chelsea", "Everton"]) == 1

print("All tests passed. ☐")
```

5.2 Next, write a function `find_nearest_smaller` that on input a list `lst` and an element `x` outputs the largest position `i` such that `L[i] <= x`. The function should output `None` if every element of `lst` is larger than `x`. For example,

```
find_nearest_smaller("Liverpool", ["Manchester United", "Arsenal",
                                     "Chelsea", "Everton"]) == 3
find_nearest_smaller("Chelsea", ["Manchester United", "Arsenal",
                                   "Chelsea", "Everton"]) == 2
find_nearest_smaller("Arsenal", ["Manchester United", "Arsenal",
                                   "Chelsea", "Everton"]) == 1

# TODO: Your code here
def find_nearest_smaller(L, P):
    pass

# Use this cell to test your code
assert find_nearest_smaller("Liverpool", ["Manchester United",
                                             "Arsenal", "Chelsea", "Everton"]) == 3
assert find_nearest_smaller("Chelsea", ["Manchester United",
                                           "Arsenal", "Chelsea", "Everton"]) == 2
assert find_nearest_smaller("Arsenal", ["Manchester United",
                                           "Arsenal", "Chelsea", "Everton"]) == 1

print("All tests passed. ☐")
```

5.3 When we have a sorted list, we can search more efficiently using binary search. Let's write `find_nearest_smaller_fast` that can compute the same value as `find_nearest_smaller` on sorted lists but uses binary search.

```
# TODO: Your code here
def find_nearest_smaller_fast(x, lst):
    pass

# Use this cell to test your code
assert find_nearest_smaller_fast(15, [2, 10, 34, 55, 100, 204]) == 1
assert find_nearest_smaller_fast(9, [1, 4, 7, 8, 9, 12, 15]) == 4
assert find_nearest_smaller_fast(10, [7, 9, 11, 13]) == 1

print("All tests passed. ☐")
```

5.4 Let's write a function `find_nearest_larger_fast` that is like `find_nearest_smaller_fast` but computes the nearest largest value instead of the smallest value.

```
# TODO: Your code here
def find_nearest_larger_fast(x, lst):
    pass

# Use this cell to test your code
assert find_nearest_larger_fast(15, [2, 10, 34, 55, 100, 204]) == 2
assert find_nearest_larger_fast(9, [1, 4, 7, 8, 9, 12, 15]) == 4
assert find_nearest_larger_fast(10, [7, 9, 11, 13]) == 2

print("All tests passed. ☐")
```

Congratulations ☐